

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D1 — Design Specification Document

Part 1 Submission

8 May 2026

Halit Batuhan ASLAN

220104004003

Design Specification Contents

Language Overview

Lexical Structure

Syntax

Names, Binding, Scope, Lifetime

Language Overview;

My language name is "Ayar", which corresponds to an old Turkish word meaning adjustment and regulation. This name directly reflects the language's purpose: to set and tune hyperparameters for machine learning models and present the user with the best option. The domain is ML model configuration, training simulation, and best-model selection. Ayar does not run real ML libraries; it simulates the training process and metric generation internally. One sample program is ;

```
// Dataset

dataset iris = load("iris.csv");

split iris into train(0.70), validation(0.15), test(0.15);

// Models

model KNN knn_small
{
    k = 3,
    distance = "euclidean"
}

model DecisionTree dt_main
{
    max_depth = 5,
    criterion = "gini"
}

// Experiment

experiment iris_exp
{
    train knn_small on iris.train;
    train dt_main on iris.train;
    evaluate knn_small on iris.validation;
    evaluate dt_main on iris.validation;
    collect metrics [accuracy, f1];
    analyze overfitting
    {
        threshold = 0.10;
        underfit_max_acc = 0.70;
```

```

}

compare by f1 higher_is_better;

exclude if overfit or underfit;

select best;

}

// Final evaluation and report

evaluate iris_exp.best on iris.test;

report summary

{

    metrics = [accuracy, f1],

    show = overfitting_analysis

}

```

My language prioritizes readability (Sebesta §1.3.1) because I believe reading and reviewing code later is far more important than writing it quickly. For cost (§1.3.4), readability directly reduces maintenance cost — in a typical ML project lifecycle, reading and reviewing code takes significantly more time than writing it, so optimizing for readability lowers the long-term cost of using the language. For reliability (§1.3.3), Ayar enforces strong typing where `metric ≠ float`, detects data leakage (training on the test set) as a runtime error, and uses name equivalence to prevent accidental type confusion — all of which reduce silent errors in model selection. Ayar knowingly sacrifices writability (§1.3.2): keywords like `higher_is_better` and `overfitting_analysis` are verbose, and every `{` must be placed on its own line. These rules make programs longer to type but significantly easier to read and review.

Lexical Structure;

A	B
Token Categories	Regular Expression or Pattern
INT_LIT	<code>[0-9]+</code>
FLOAT_LIT	<code>[0-9]+ \. [0-9]+</code>
STRING_LIT	<code>\("[^"\n]*\"</code>
BOOL_LIT	<code>true false</code>
OPERATOR	<code>* - + / < > ==</code>
SEPARATOR	<code>{ } ; : , ()</code>
COMMENT	<code>// [^\\n]* /* [^*]* */</code>
KEYWORD	<code>experiment, model, train etc.</code>
IDENTIFIER	<code>iris, knn_baseline etc.</code>

Syntax;

<program> ::= { <top_level_decl> }

<top_level_decl> ::= <dataset_decl> | <split_stmt> | <model_decl> | <experiment_decl> |
<eval_stmt> | <report_stmt>

(* — DATASET — *)

<dataset_decl> ::= "dataset" <identifier> "=" "load" "(" <string_literal> ")" ";"

<split_stmt> ::= "split" <identifier> "into" <split_part> "," <split_part> "," <split_part> ";"

<split_part> ::= <split_role> "(" <float_literal> ")"

<split_role> ::= "train" | "validation" | "test"

(* — MODEL — *)

<model_decl> ::= "model" <model_type> <identifier> "{" <field_list> "}"

<model_type> ::= "KNN" | "DecisionTree" | "SVM" | "NaiveBayes" | "LogisticRegression"

<field_list> ::= <field> { "," <field> }

<field> ::= <identifier> "=" <field_value>

<field_value> ::= <int_literal> | <float_literal> | <string_literal> | <bool_literal>

(* — EXPERIMENT — *)

<experiment_decl> ::= "experiment" <identifier> "{" { <experiment_stmt> } "}"

<experiment_stmt> ::= <train_stmt> | <evaluate_stmt> | <collect_stmt> | <analyze_stmt> |
<compare_stmt> | <exclude_stmt> | <select_stmt>

<train_stmt> ::= "train" <identifier> "on" <dataset_ref> ";"

<evaluate_stmt> ::= "evaluate" <identifier> "on" <dataset_ref> ";"

<dataset_ref> ::= <identifier> "." <split_role>

<collect_stmt> ::= "collect" "metrics" "[" <metric_list> "]" ";"

<metric_list> ::= <metric_literal> { "," <metric_literal> }

<analyze_stmt> ::= "analyze" "overfitting" "{" <analyze_field_list> "}"

<analyze_field_list> ::= <analyze_field> { <analyze_field> }

<analyze_field> ::= "threshold" "=" <float_literal> ";" | "underfit_max_acc" "=" <float_literal> ";"

<compare_stmt> ::= "compare" "by" <metric_literal> <compare_direction> ";"

<compare_direction> ::= "higher_is_better" | "lower_is_better"

<exclude_stmt> ::= "exclude" "if" <exclude_condition> ";"

<exclude_condition> ::= <exclude_term> { "or" <exclude_term> }

<exclude_term> ::= "overfit" | "underfit"

<select_stmt> ::= "select" "best" ";"

(* — TOP-LEVEL EVALUATE — *)

<eval_stmt> ::= "evaluate" <eval_target> "on" <dataset_ref> ";"

<eval_target> ::= <identifier> | <identifier> "." <identifier> | "[" <identifier> "." <identifier> "]"

(* — REPORT — *)

<report_stmt> ::= "report" <identifier> "{" <report_field_list> "}"

<report_field_list> ::= <report_field> { "," <report_field> }

<report_field> ::= "metrics" "=" "[" <metric_list> "]" | "show" "=" <show_value>

<show_value> ::= "overfitting_analysis" | <string_literal>

(* — EXPRESSIONS — *)

<expr> ::= <and_expr> | <expr> "or" <and_expr>

<and_expr> ::= <not_expr> | <and_expr> "and" <not_expr>

<not_expr> ::= <rel_expr> | "not" <not_expr>

<rel_expr> ::= <add_expr> | <add_expr> <rel_op> <add_expr>

<rel_op> ::= "==" | "!=" | "<" | "<=" | ">" | ">="

<add_expr> ::= <mul_expr> | <add_expr> "+" <mul_expr> | <add_expr> "-" <mul_expr>

<mul_expr> ::= <unary_expr> | <mul_expr> "*" <unary_expr> | <mul_expr> "/" <unary_expr>

<unary_expr> ::= <primary> | "-" <unary_expr>

<primary> ::= <int_literal> | <float_literal> | <bool_literal> | <string_literal> | <metric_literal>
| <identifier> "." <identifier> | <identifier> | "(" <expr> ")"

(* — LEXICAL — *)

<identifier> ::= <letter> { <letter> | <digit> | "_" }

<int_literal> ::= <digit> { <digit> }

<float_literal> ::= <digit> { <digit> } "." { <digit> } | "." <digit> { <digit> }

<bool_literal> ::= "true" | "false"

<string_literal> ::= "" { <string_char> } ""

<metric_literal> ::= "accuracy" | "precision" | "recall" | "f1"

<letter> ::= "a" | ... | "z" | "A" | ... | "Z"

<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<string_char> ::= (* any char except "" and unescaped newline *)

Names, Binding, Scope, Lifetime;

Legal Identifiers. In “ayar”, an identifier must begin with a letter (a–z or A–Z) and may be followed by any combination of letters, digits, and underscores. Underscores are not permitted as the first character — this is a deliberate restriction to keep identifiers visually clean and to avoid confusion with internal naming conventions. There is no length limit on identifiers. Keywords such as `dataset`, `model`, `train`, and `evaluate` are reserved words and cannot be used as user identifiers; the lexer classifies them before they reach the parser (Sebesta §5.2, reserved words approach). The same applies to metric literals (`accuracy`, `precision`, `recall`, `f1`) and boolean literals (`true`, `false`), which are recognized as dedicated token types rather than identifiers.

Binding and Binding Time. “ayar” distinguishes between bindings that occur at language design time and those that occur at runtime. The types of operators (`+`, `-`, `*`, `/`) and the set of valid model types (`KNN`, `DecisionTree`, `SVM`, `NaiveBayes`, `LogisticRegression`) are bound at language design time — they are fixed in the grammar and cannot be extended by the user. The association between a model name and its hyperparameter values (e.g., `knn_clf` bound to `{k = 3, distance = "euclidean"}`) occurs at runtime, specifically at elaboration time when the model declaration is processed. Similarly, the binding of a dataset name to its loaded file path happens at runtime when the dataset declaration is executed. In Sebesta's terminology (§5.4), these are dynamic bindings: the binding exists only during program execution and is established when the declaration is encountered during interpretation.

Scoping: Static (Lexical). “ayar” uses static scoping (Sebesta §5.7). There are exactly two scope levels in the current design:

- **Global scope:** All dataset, model, and experiment declarations live in global scope. A model declared at the top level is visible to every experiment block in the program.
- **Experiment scope:** The body of an experiment block creates its own inner scope. Statements inside the experiment (such as `train`, `evaluate`, `collect`, `analyze`, `compare`, `exclude`, `select`) can reference models and datasets from the global scope, but any names introduced inside the experiment (such as the implicit best result after `select best`) are local to that experiment.

The choice of static scoping is motivated by predictability and reliability (Sebesta §1.3.4). In an ML configuration language, the user must know exactly which model a `train` statement refers to by reading the source code — not by tracing the runtime call history. If Ayar used dynamic scoping instead, a model name inside an experiment could accidentally resolve to a different model depending on the order of execution, which would silently change training results. This would be catastrophic in an ML context where reproducibility is essential. For example, if two experiment blocks both declared a local model named `m`, dynamic scoping could cause the second experiment's `train m` to accidentally pick up the first experiment's `m` if scopes were not properly isolated. Static scoping eliminates this entire class of errors.

Lifetime. Local variables in Ayar — specifically, the internal state of an experiment block (trained model results, collected metrics, the best selection) — have stack-dynamic lifetime (Sebesta §5.8.2). They are allocated when the experiment block begins execution and deallocated when the block ends. This means experiment results do not persist after the block closes, unless explicitly referenced by a top-level `evaluate` or `report` statement through dotted

access (e.g., `iris_exp.best`). Global declarations (datasets and models) have static lifetime — they exist for the entire duration of the program, from the moment they are declared until the program terminates. There are no explicit-heap-dynamic variables in Ayar; the language does not provide manual memory allocation or deallocation, which is consistent with its goal of being a high-level DSL that abstracts away low-level concerns.